

Katedra informatiky
Přírodovědecká fakulta
Univerzita Palackého v Olomouci

BAKALÁŘSKÁ PRÁCE

Paralelní Programování v Go



2026

Vedoucí práce:
Mgr. Jan Laštovička, Ph.D.

Richard Bebjak

Studijní program: Informatika,
Specializace: Programování a vývoj
software

Bibliografické údaje

Autor: Richard Bebjak
Název práce: Paralelní Programování v Go
Typ práce: bakalářská práce
Pracoviště: Katedra informatiky, Přírodovědecká fakulta, Univerzita Palackého v Olomouci
Rok obhajoby: 2026
Studijní program: Informatika, Specializace: Programování a vývoj software
Vedoucí práce: Mgr. Jan Laštovička, Ph.D.
Počet stran: 17
Přílohy: elektronická data v úložišti katedry informatiky
Jazyk práce: slovenský

Bibliographic info

Author: Richard Bebjak
Title: Paralel programming in Go
Thesis type: bachelor thesis
Department: Department of Computer Science, Faculty of Science, Palacký University Olomouc
Year of defense: 2026
Study program: Computer Science, Specialization: Programming and Software Development
Supervisor: Mgr. Jan Laštovička, Ph.D.
Page count: 17
Supplements: electronic data in the storage of department of computer science
Thesis language: Slovak

Anotácia

Synopsis

Kľúčové slová:

Keywords:

Děkuji, děkuji, děkuji.

Odevzdaním tohto textu jeho autor/ka cestne vyhlasuje, že celú prácu vrátane príloh vypracoval/a samostatne a za použitia iba zdrojov spomínaných v texte práce a uvedených v zozname literatúry.

Obsah

1	Čo je to Communication Sequential Process	7
1.1	Výrobca a spotrebiteľ CSP	7
2	Goroutine	8
2.1	Model a plánovač goroutine	8
3	Komunikačné kanály	9
3.1	Bufferované a nebufferované komunikačné kanály	10
3.2	Jednosmerný komunikačný kanál	10
4	Synchronizačné primitíva	10
4.1	WaitGroup	10
4.2	Mutex a RWMutex	11
5	Select	13
6	Chat server	13
6.1	Klientské spojenie	13
6.2	Klientské požiadavky	14
6.3	Go pamäťový model	15
7	Chat server	15
7.1	Funkčné požiadavky	15
7.2	Súbežná archytekúra	15
8	Paralelné algoritmy	15
9	Paralené programovanie	15
9.1	Paralelné a súbežné programovanie	15
9.2	Problémy súbežného programovania	15
10	Jazyk Go	15
10.1	Goroutines	15
	Literatúra	17

Zoznam tabuliek

1 Čo je to Communication Sequential Process

Communication Sequential Process (CSP) je základný teoretický prístup ku súbežnému programovaniu.^[1] Tento prístup definuje ako môžu súbežne bežiacie procesy spolupracovať a komunikovať. Na rozdiel od tradičných prístupov založených na zdialanej pamäti, CSP je založené na komunikácii medzi procesmi ako hlavným mechanizmom koordinácie. Procesy si neposkytujú priamy prístup k pamäti, čím sa znižuje počet chýb súbehu. Základnou myšlienkou CSP je, že program sa skladá z viacerých sekvečných procesov vykonávaných súbežne, ktoré medzi sebou komunikujú prostredníctvom vstupno-výstupných operácií.

V CSP sú vstup a výstup základné operácie, na rozdiel od tradičného prístupu, kde je základnou operáciou priradenie. Tieto operácie umožňujú procesom medzi sebou komunikovať a koordinovať svoju činnosť. CSP umožňuje definovať paralelné vykonávanie viacerých procesov pomocou paralelného príkazu. Všetky procesy sa vykonávajú súčasne a program pokračuje až po ich dokončení. Komunikácia medzi procesmi v CSP je synchronná, čo znamená, že odosielateľ aj príjemca musia byť pripravení na komunikáciu v rovnakom čase. Neexistuje žiadne implicitné bufferovanie správ, čím sa zabezpečuje deterministické správanie komunikácie. Model CSP predpokladá daný počet procesov. Nepodporuje dynamické vytváranie procesov ani rekurziu, čo zjednodušuje model.

1.1 Výrobca a spotrebiteľ CSP

CSP sa dá v tomto príklade chápať ako monitor s ktorým komunikujú viaceré procesy. Procesy musia byť odlišne pomenované napr. „Výrobca“ a „Spotrebiteľ“, alebo odlišný index napríklad „X(0)“. Každá komunikácia musí mať jednoznačne určený svoj zdroj alebo cieľ. Keď je monitor pripravený komunikovať s ktorýmkoľvek používateľským procesom, zväčša ktorý príde prvý, používa strážený príkaz s rozsahom.

$$[(i : 1..100); X(i)?(value\ parameters) \rightarrow \dots; X(i)!(results);]$$

Premenná „i“ sa používa na posielanie výsledkov späť procesu, ktorý komunikáciu inicioval.

Ako monitor nie je pripravený na komunikáciu od konkrétneho procesu napríklad „X(j)“, môže byť vstupný príkaz doplnený o booleovskú podmienku, tak zvanú stráž. Napríklad dve po sebe idúce požiadavky od rovnakého procesu, môžu byť zablokovanie pomocou podmienky „j=0“.

$$*[(i : 1..100)i \neq j; X(i)?(parameters) \rightarrow \dots; i := j;]$$

V tomto prípade bude výstup procesu „X(j)“ odložený, kým nebude spracovaná požiadavka iného procesu. Podobne je možné použiť podmienky na oddialenie prijatia vstupov, ktoré by porušili plánovacie obmedzenia. ^[1]

2 Goroutine

Goroutine je funkcia ktorá sa vykonáva súbežne. Nie sú to vlákna operačného systému ani vlákna riadene runtime prostredím. Ide o vyššiu abstrakciu nazývanú korutiny. Korutiny sú súbežné podprogramy, ktoré nemôžu byť prerušené zvonka. Obsahujú však body, v ktorých môžu byť pozastavené a znovu spustené. Na rozdiel od vláken je prepínanie medzi gorutinami rýchlejšie približne o 1,242 μ s.^[2] Goroutine sa dajú spustiť pomocou kľúčového slova `go`.

```
func main{
    go SayHello()
}

func SayHello(){
    fmt.Println("Hello")
}
```

Je možné goroutine implementovať ako anonymné funkcie.

```
go func(){
    fmt.Println("Hello")
}
```

Ako anonymné funkcie musia byť zavolané okamžite. Vieme goroutine priradiť do premennej.

```
sayHello := func(){
    fmt.Println("Hello")
}

go sayHello()
```

Goroutine sú špecifické pre jazyk Go, pretože sú integrované do runtime prostredia Go. Goroutine si sami neurčujú body pozastavenia. Runtime Go sleduje ich správanie a automaticky ich pozastaví, a znovu spustí, keď sú opätovne pripravené pokračovať. Goroutine bežia súbežne, čo nie je ich vlastnosťou, preto potrebujú mechanizmus, ktorý im umožní bežať a vykonávať sa súbežne.

2.1 Model a plánovač goroutine

Go používa plánovač „M:N“ a model súbežnosti fork-join.^[2] Teda na „M“ vláknach sa vykonáva „N“ goroutine. Goroutine sú mapované na tieto vlákna a go runtime sa stará o ich rozdelenie a efektívne vykonávanie. Stará sa o ich spúšťanie, ak je nejaká goroutine blokováaná tak go runtime môže spustiť inú.

Podľa modelu fork-join, „Fork“ znamená, že program môže vytvoriť novú goroutine, ktorá sa vykonáva súbežne s rodičovským kódom. A „Join“ znamená, že sa tieto vetvy spoja v mieste nazývanom join point. ² Spájanie vetiev nie je funkciou programu, preto je nutná synchronizácia pri goroutine.

3 Komunikačné kanály

Komunikačné kanály predstavujú jeden zo základných mechanizmov synchronizácie a komunikácie medzi goroutines. Sú zabudované referečným typom, ktorý umožňuje bezpečný prenos dát. Ich návrh vychádza z Communicating Sequential Processes, ktorý zdôrazňuje komunikáciu medzi procesmi miesto zdieľania pamäte.^[2]

Komunikačné kanály sa deklarujú pomocou kľúčového slova `chan` a vždy sa deklarujú aj s určitým typom. To znamená, že kanál môže prenášať hodnoty iba tohto typu. Komunikačné kanály patria k referečným typom, preto ich deklarácia nevytvára potrebnú pamäťovú štruktúru. Pamäť sa alokuje pomocou zabudovanej funkcie `make`.

```
make(chan type, buffer_size)
```

Takto vytvorený komunikačný kanál umožňuje zapisovanie a čítanie hodnôt. Zápis do komunikačného kanála umožňuje operátor `<-`, ktorý sa umiestňuje na pravú stranu názvu komunikačného kanála, pre čítanie sa operátor umiestňuje na ľavú stranu názvu komunikačného kanála.

```
channel <- data
variable = <- channel
```

Dôležitou vlastnosťou komunikačného kanála je jeho blokujúce správanie. Ak goroutine zapisuje do komunikačného kanála, ktorý je plný, je jej proces blokovaný kým iná goroutine z komunikačného kanála hodnotu neprečíta. Podobne čítanie z prázdneho komunikačného kanála je blokovaný kým nie je do komunikačného kanála zapísaná nová hodnota.

Komunikačný kanál je možné zavrieť pomocou funkcie `close`. Pri čítaní z kanála je možné získať druhú návratovú hodnotu typu `bool`, ktorá indikuje jeho otvorenie. Uzavretie teda signalizuje príjmateľovi, že do komunikačného kanála nebude viac zapisované. Ak odosielateľ pošle data do uzavretého komunikačného kanálu, príjmateľ obdrží `error send on closed channel`. Nasledujúci príklad demonštruje použitie.

```
1    var channel = make(chan int, 1)
2
3    go func(){
4        channel <- 42
5    }()
6
7    result := <- channel
8    close(channel)
```

Na riadku jedna si vytvoríme komunikačný kanál s referečným typom `int` a veľkosťou buffera jedna. Na riadku tri spustíme novú goroutine. Na riadku štyri v goroutine uložíme do komunikačného kanála hodnotu 42. Na riadku 7 hlavné vlákno čaká na data z komunikačného kanála. Ak dáta prídu, uloží do premen-

nej „result“ data z komunikačného kanála, v tomto prípade 42. Na riadku 8 uzavrieme komunikačný kanál.

3.1 Bufferované a nebufferované komunikačné kanály

Každý komunikačný kanál môže byť bufferovaný alebo nebufferovaný. Nebufferovaný komunikačný kanál nemá žiadnu kapacitu a každá operácia zápisu musí byť okamžite spravovaná operáciou čítania.

Bufferovaný komunikačný kanál sa definuje pomocou voliteľného druhého argumentu funkcie `make`. Bufferovaný komunikačný kanál obsahuje bugger s definovanou kapacitou, ktorá umožní uložiť určitý počet hodnôt do buffera bez okamžitého čítania. Bufferovaný komunikačný kanál zablokuje zápis až v momente keď je buffer plný.

3.2 Jednosmerný komunikačný kanál

Jazyk Go umožňuje definovanie jednosmerných kanálov, ktoré podporujú iba zápis alebo čítanie. Tieto komunikačné kanály sa používajú ako parametre funkcií, aby určili akým spôsobom sa komunikačný kanál v danej časti programu používa. Tento mechanizmus zvyšuje bezpečnosť programu, pretože kompilátor dokáže zachytiť nesprávne použitie.

4 Synchronizačné primitíva

Jazyk Go poskytuje `sync` balík, ktorý obsahuje synchronizačné primitíva. Tieto synchronizačné primitíva sú používané pri nízkoúrovňovom prístupe do pamäte. Rozdiel medzi ostatnými jazykmi a jazykom Go je, že Go poskytuje novú sadu synchronizačných primitív pre súbežnosť nad sady primitív pre synchronizáciu prístupu k pamäti.

4.1 WaitGroup

Je to nástroj z balíka `sync`, ktorého účelom je umožniť programu počkať na dokončenie skupiny goroutine pred tým, než bude pokračovať. Mechanizmus sa využíva v prípade, že sa vykonáva viacero nezávislých úloh. `WaitGroup` si môžeme predstaviť ako bezpečný čítač. Tento čítač eviduje počet goroutine, ktoré ešte nedokončili svoju činnosť. Čítač sa zvyšuje pomocou metódy `Add(n)`, kde „n“ udáva počet goroutine, ktoré sa majú vykonať. Čítač sa znižuje pomocou metódy `Done()`. Program je zablokovaný na metóde `Wait()`, kým hodnota čítača nedosiahne nulovú hodnotu. Nasledujúci príklad demonštruje použitie.

```
1     var wg sync.WaitGroup
2
3         for i := 0; i < 10; i++ {
4             wg.Add(1)
```

```

5             go func(i int) {
6                 defer wg.Done()
7                 fmt.Println("Goroutine", i)
8                 time.Sleep(200)
9             }(i)
10        }
11        wg.Wait()
12        fmt.Println("End")

```

Na prvom riadku si definujeme WaitGroup. Následne v cykle na riadku tri zvýšime hodnotu WaitGroup o jedna. Na riadku štyri pustíme novú goroutine. V goroutine na riadku päť zabezpečíme, že po dokončení činnosti, zníži hodnotu WaitGroup o jedna. Následne vykoná vypís a čakanie pre simuláciu paralelného výpočtu. Na riadku jedenásť program čaká, kým hodnota WaitGroup bude nulová. následne vypíše „End“.

4.2 Mutex a RWMutex

Mutex to nástroj z balíka sync, ktorého účelom je chrániť časť programu so zdieľanými zdrojmi, ku ktorým môže pristupovať iba jedna goroutine. Pred vstupom do časti programu so zdieľanými zdrojmi, musí goroutine získať zámok pomocou metódy `Lock()`. Po ukončení činnosti v danej časti programu, uvoľní zámok pomocou metódy `Unlock()`. Počas držania zámku, nemôže iná goroutine vstúpiť do tejto časti programu. RWMutex rozširuje Mutex o zámok, ktorý rozlišuje operácie čítania a zápisu. Metódy na získanie zámku `RLock()` / `Lock()`, metódy na uvoľnenie zámku `RUnlock()` / `Unlock()`. RWMutex umožňuje získať zámok pre čítanie viacerým goroutine, ale pre zápis umožňuje získať zámok jednej goroutine. Počas zápisu nie je umožnené získať zámok pre čítanie. Pri zápise čaká kým sú uvoľnené všetky zámky. Nasledujúci príklad demonštruje použitie.

```

1    var wg sync.WaitGroup
2    var rw sync.RWMutex
3
4    observer := func(i int) {
5        defer wg.Done()
6        rw.RLock()
7        fmt.Println("Reading started", i)
8        time.Sleep(400 * time.Millisecond)
9        fmt.Println("Reading ends", i)
10       rw.RUnlock()
11    }
12
13    writer := func(i int) {
14        defer wg.Done()
15        defer rw.Unlock()
16        rw.Lock()

```

```

17         fmt.Println("Writting started", i)
18         time.Sleep(400 * time.Millisecond)
19         fmt.Println("Writting ended", i)
20     }
21
22     for i := 0; i < 100; i++ {
23         wg.Add(1)
24         if i%25 == 0 {
25             go writer(i)
26         } else {
27             go observer(i)
28         }
29     }
30     wg.Wait()
31     fmt.Println("End")

```

Na riadku dva si definujeme RWMutex. Na riadku štyry si definujeme funkciu čitateľa, ktorá sa pokúsi získať zámok pre čítanie vypíše, že začala, počká, vypíše, že skončila a po skončení uvoľní zámok pre čítanie. Na riadku trinásť si definujeme funkciu zapisovateľa, ktorá sa pokúsi získať zámok, vypíše, že začala, počká, vypíše, že skončila a po skončení uvoľní zámok. Na riadku dvadsaťdva je cyklus, v ktorom voláme funkciu zapisovateľa ak „i“ modulo 25 je nula, inak voláme funkciu čitateľa. Následne počkáme na dokončenie všetkých goroutine a vypíšeme že sme skončili. Výsledok tohto kódu bude približne.

```

"Writting started 0"
"Writting ended 0"
"Reading started 1"
"Reading started 2"
"Reading started 5"
...
"Reading ended 98"
"Reading ended 97"
"Reading ended 99"
"Writting started 50"
"Writting ended 50"
"Writting started 75"
"Writting ended 75"

```

Z tohto výsledku vidíme, že RWMutex uprednostňuje čitateľov. Vynútenie zámku pre zapisovateľa je možné pomocou ďalších synchronizačných nástrojov ako napríklad pomocou komunikačných kanálov.

5 Select

6 Chat server

Navrhnutý prototyp servru je založený na architektúre klient-server, kde server zabezpečuje výmenu správ medzi pripojenými klientmi. Komunikácia medzi klientom a serverom je realizovaná cez Transmission Control Protocol (TCP), ktorý zabezpečuje spoľahlivý prenos dát, zachovanie poradia správ a detekciu chýb pri prenose. Server je implementovaný v programovacom jazyku Go, využívajúc jeho vstavanú podporu pre súbežné programovanie prostredníctvom goroutine. Hlavná úloha servera spočíva v prijímaní nových klientských spojení, spracovaní ich požiadavkov a správe komunikačných miestností.

6.1 Klientské spojenie



Obr. 1: Obrázok číslo 2

Pri spustení server vytvorí TCP listener na porte 8080 pomocou funkcie `net.Listen`. Následne v nekonečnom cykle čaká na nové pripojenie, ak je pripojenie úspešné tak vytvorí pre to spojenie novú goroutine. Týmto je zabezpečené, že každý klient je obslužený súbežne a bez blokovania prijímania ďalších klientov.

Každý klient je obslužený na samostatnej goroutine, ktorá vykonáva funkciu `HandleConnection`. Toto riešenie predstavuje z pohľadu architektúry model:

- jeden hlavný serverový proces
- jedna goroutine na klienta

Riešenie je efektívne, pretože goroutine sú ľahšie ako klasické vlákna a umožňujú vysokú mieru škálovateľnosti.

6.2 Klientské požiadavky

```
user := strings.TrimSpace(reader.Text())
for reader.Scan() {
    line := strings.TrimSpace(reader.Text())

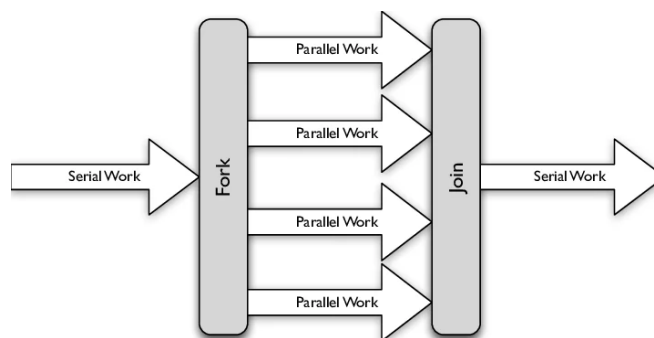
    if chatLog == nil {
//pripojenie do komunikacnej miestnosti
        continue
    }

    switch line {
        case "":
            continue
        case "/fetch":
//vyziadanie novych sprav z komunikacnej miestnosti
        case "/fetchAll":
//vyziadanie sprav zo vsetkych navstivenych komunikacnych miestnosti
        case "/broadcast":
//poslanie sprav vsetkym navstivenym komunikacnym miestnostiam
        case "exit":
//opustenie komunikacnej miestnosti
        case "logout":
//odhlasenie zo serveru
        default:
//poslanie sprav
    }
}
```

Po pripojení je od klienta vyžiadané meno. Následne v nekonečnom cykle prvá podmienka pripojí klienta do komukačnej miestnosti a v následných cykloch spracúva požiadavky:

1. /fetch
2. /fetchAll
3. /broadcast
4. exit
5. logout
6. default

- 6.3 Go pamäťový model
- 7 Chat server
 - 7.1 Funkčné požiadavky
 - 7.2 Súbežná architektúra
- 8 Paralelné algoritmy
- 9 Paralelné programovanie
 - 9.1 Paralelné a súbežné programovanie
 - 9.2 Problémy súbežného programovania
- 10 Jazyk Go
 - 10.1 Goroutines



Obr. 2: Text pod obrazkom

Literatúra

- [1] Hoare, C. A. R. Communicating Sequential Processes. In Graham, S. L.; Rivest, R. L. (ed.). *Programming Techniques*. New York: Academic Press, 1978.
- [2] Cox-Buday, Katherine. *Concurrency in Go*. First. Sebastopol, CA: O'Reilly Media, 2017. 224 str. ISBN 978-1-491-94119-5.
- [3] Beckingsale, David. *Towards Scalable Adaptive Mesh Refinement on Future Parallel Architectures*. 2015.